Lesson 3: Likelihood-based Inference for POMP Models

Aaron A. King Edward L. Ionides Kunyang He

Saturday, March 14, 2026

This lesson develops likelihood-based inference for POMP models, with a focus on the particle filter algorithm for computing the likelihood.

Objectives

Students completing this lesson will:

1. Gain an understanding of the nature of the problem of likelihood computation for POMP models.
2. Be able to explain the simplest particle filter algorithm.
3. Gain experience in the visualization and exploration of likelihood surfaces.
4. Be able to explain the tools of likelihood-based statistical inference that become available given numerical accessibility of the likelihood function.

Overview

Conceptual links in our methodological approach

- The Monte Carlo technique called the **particle filter** is central for connecting the higher-level ideas of POMP models and likelihood-based inference to the lower-level tasks involved in carrying out data analysis.

- We employ a standard toolkit for likelihood-based inference:
  - Maximum likelihood estimation
  - Profile likelihood confidence intervals
  - Likelihood ratio tests for model selection
  - Other likelihood-based model comparison tools such as AIC
- We seek to better understand these tools, and to figure out how to implement and interpret them in the specific context of POMP models.

The Likelihood Function

- The basis for modern frequentist, Bayesian, and information-theoretic inference.
- Method of maximum likelihood introduced by (Fisher, 1922).

- The likelihood function itself is a representation of what the data have to say about the parameters.

Definition of the likelihood function:

- Data are a sequence of N observations, denoted $y^*_{1:N}$.
- A statistical model is a density function $f_{Y_{1:N}}(y_{1:N};\theta)$ which defines a probability distribution for each value of a parameter vector θ.
- To perform statistical inference, we must decide, among other things, for which (if any) values of θ it is reasonable to model $y^*_{1:N}$ as a random draw from $f_{Y_{1:N}}(y_{1:N};\theta)$.

The likelihood function is:

$$\mathcal{L}(\theta) = f_{Y_{1:N}}(y^*_{1:N};\theta),$$

the density function evaluated at the data.

It is often convenient to work with the **log-likelihood function**:

$$\ell(\theta) = \log \mathcal{L}(\theta) = \log f_{Y_{1:N}}(y^*_{1:N};\theta).$$

A Simulator is Implicitly a Statistical Model

- For simple statistical models, we may describe the model by explicitly writing the density function $f_{Y_{1:N}}(y_{1:N};\theta)$. One may then ask how to simulate a random variable $Y_{1:N} \sim f_{Y_{1:N}}(y_{1:N};\theta)$.
- For many dynamic models it is much more convenient to define the model via a procedure to **simulate** the random variable $Y_{1:N}$. This implicitly defines the corresponding density $f_{Y_{1:N}}(y_{1:N};\theta)$.
- For a complicated simulation procedure, it may be difficult or impossible to write down or even compute $f_{Y_{1:N}}(y_{1:N};\theta)$ exactly.
- It is important to bear in mind that **the likelihood function exists even when we don't know what it is!**

The Likelihood for a POMP Model

Recall the structure of a POMP model:

- Measurements, Y_n, at time t_n depend on the latent process, X_n, at that time.
- The Markov property asserts that latent process variables depend on their value at the previous timestep.
- The distribution of X_{n+1}, conditional on X_n, is independent of X_k for $k < n$ and Y_k for $k \leq n$.
- The distribution of Y_n, conditional on X_n, is independent of all other variables.

The joint density factors as:

$$f_{X_{0:N},Y_{1:N}}(x_{0:N}, y_{1:N}; \theta) = f_{X_0}(x_0; \theta) \prod_{n=1}^{N} f_{X_n|X_{n-1}}(x_n|x_{n-1}; \theta)\, f_{Y_n|X_n}(y_n|x_n; \theta).$$

The marginal density for the sequence of measurements, $Y_{1:N}$, evaluated at the data, $y^*_{1:N}$, is:

$$\mathcal{L}(\theta) = f_{Y_{1:N}}(y^*_{1:N}; \theta) = \int f_{X_{0:N},Y_{1:N}}(x_{0:N}, y^*_{1:N}; \theta)\, dx_{0:N}.$$

This integral is **high dimensional** and, except for the simplest cases, cannot be reduced analytically.

Monte Carlo Likelihood by Direct Simulation

- First, let's rewrite the likelihood integral using an equivalent factorization:

$$\mathcal{L}(\theta) = f_{Y_{1:N}}(y^*_{1:N}; \theta) = \int \left\{ \prod_{n=1}^{N} f_{Y_n|X_n}(y^*_n|x_n; \theta) \right\} f_{X_{0:N}}(x_{0:N}; \theta)\, dx_{0:N}.$$

- Notice that the likelihood can be written as an expectation:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\prod_{n=1}^{N} f_{Y_n|X_n}(y^*_n|X_n; \theta) \right],$$

  where the expectation is taken with $X_{0:N} \sim f_{X_{0:N}}(x_{0:N}; \theta)$.

- Using a law of large numbers, we can approximate:

$$\mathcal{L}(\theta) \approx \frac{1}{J} \sum_{j=1}^{J} \prod_{n=1}^{N} f_{Y_n|X_n}(y^*_n|X^j_n; \theta),$$

  where $\{X^j_{0:N}, j = 1, \dots, J\}$ is a Monte Carlo sample drawn from $f_{X_{0:N}}(x_{0:N}; \theta)$.

Problems with this naive approach:

- This scales poorly with dimension. It requires Monte Carlo effort that scales **exponentially** with the length of the time series.
- Once a simulated trajectory diverges from the data, it will seldom come back.
- Simulations that lose track of the data make negligible contributions to the likelihood estimate.

The Particle Filter

Fortunately, we can compute the likelihood for a POMP model by a much more efficient algorithm.

We proceed by factorizing the likelihood differently:

$$\mathcal{L}(\theta) = f_{Y_{1:N}}(y^*_{1:N};\theta) = \prod_{n=1}^{N} f_{Y_n|Y_{1:n-1}}(y^*_n|y^*_{1:n-1};\theta)$$

$$= \prod_{n=1}^{N} \int f_{Y_n|X_n}(y^*_n|x_n;\theta)\, f_{X_n|Y_{1:n-1}}(x_n|y^*_{1:n-1};\theta)\, dx_n.$$

The prediction formula (from Markov property):

$$f_{X_n|Y_{1:n-1}}(x_n|y^*_{1:n-1};\theta) = \int f_{X_n|X_{n-1}}(x_n|x_{n-1};\theta)\, f_{X_{n-1}|Y_{1:n-1}}(x_{n-1}|y^*_{1:n-1};\theta)\, dx_{n-1}.$$

The filtering formula (from Bayes' theorem):

$$f_{X_n|Y_{1:n}}(x_n|y^*_{1:n};\theta) = \frac{f_{Y_n|X_n}(y^*_n|x_n;\theta)\, f_{X_n|Y_{1:n-1}}(x_n|y^*_{1:n-1};\theta)}{\int f_{Y_n|X_n}(y^*_n|u_n;\theta)\, f_{X_n|Y_{1:n-1}}(u_n|y^*_{1:n-1};\theta)\, du_n}.$$

This suggests we keep track of two key distributions at each time t_n:

- The **prediction distribution**: $f_{X_n|Y_{1:n-1}}(x_n|y^*_{1:n-1})$
- The **filtering distribution**: $f_{X_n|Y_{1:n}}(x_n|y^*_{1:n})$

The particle filter uses Monte Carlo techniques to sequentially estimate these integrals. Hence, the alternative name of **sequential Monte Carlo (SMC)**.

Basic Particle Filter Algorithm

1. Suppose $X^F_{n-1,j}$, $j = 1, \dots, J$ is a set of J points drawn from the filtering distribution at time t_{n-1}.
2. We obtain a sample $X^P_{n,j}$ from the prediction distribution at time t_n by simply **simulating** the process model:
$$X^P_{n,j} \sim \text{process}(X^F_{n-1,j}, \theta), \quad j = 1, \dots, J.$$
3. Having obtained $X^P_{n,j}$, we obtain a sample from the filtering distribution at time t_n by **re-sampling** from $\{X^P_{n,j}, j \in 1:J\}$ with weights:
$$w_{n,j} = f_{Y_n|X_n}(y^*_n|X^P_{n,j};\theta).$$
4. The conditional likelihood
$$\mathcal{L}_n(\theta) = f_{Y_n|Y_{1:n-1}}(y^*_n|y^*_{1:n-1};\theta)$$
is approximated by:
$$\hat{\mathcal{L}}_n(\theta) \approx \frac{1}{J}\sum_j f_{Y_n|X_n}(y^*_n|X^P_{n,j};\theta).$$

5. We iterate this procedure through the data, one step at a time, alternately simulating and resampling, until we reach $n = N$.
6. The full log-likelihood then has approximation:

$$\ell(\theta) = \log \mathcal{L}(\theta) = \sum_n \log \mathcal{L}_n(\theta) \approx \sum_n \log \hat{\mathcal{L}}_n(\theta).$$

Block Diagram of Particle Filter

Particle Filter Steps

1. **Initialize**: `rinit`
2. **Predict**: `rproc` (simulate forward)
3. **Weight**: `dmeas` (evaluate measurement density)
4. **Filter**: resample particles according to weights
5. Repeat steps 2-4 for N observations

The particle filter provides an **unbiased** estimate of the likelihood. This implies a consistent but biased estimate of the log-likelihood.

Import Required Packages

```
import jax.numpy as jnp
import jax
import pandas as pd
import numpy as np
import pypomp as pp
import matplotlib.pyplot as plt
import time
```

Load Data and Build Model

```
# Load local data
meas = (pd.read_csv("Measles_Consett_1948.csv")
          .loc[:, ["week", "cases"]]
          .rename(columns={"week": "time", "cases": "reports"})
          .set_index("time")
          .astype(float))

ys = meas.copy()
ys.columns = pd.Index(["reports"])
```

Helper Functions

```python
def nbinom_logpmf(x, k, mu):
    """Log PMF of NegBin(k, mu) that is robust when mu == 0."""
    x = jnp.asarray(x)
    k = jnp.asarray(k)
    mu = jnp.asarray(mu)
    logp_zero = jnp.where(x == 0, 0.0, -jnp.inf)
    safe_mu = jnp.where(mu == 0.0, 1.0, mu)
    core = (jax.scipy.special.gammaln(k + x)
            - jax.scipy.special.gammaln(k)
            - jax.scipy.special.gammaln(x + 1)
            + k * jnp.log(k / (k + safe_mu))
            + x * jnp.log(safe_mu / (k + safe_mu)))
    return jnp.where(mu == 0.0, logp_zero, core)

def rnbinom(key, k, mu):
    """Sample from NegBin(k, mu) via Gamma-Poisson mixture."""
    key_g, key_p = jax.random.split(key)
    lam = jax.random.gamma(key_g, k) * (mu / k)
    return jax.random.poisson(key_p, lam)
```

SIR Model Components

```python
def rinit(theta_, key, covars, t0):
    """Initial state simulator for SIR model."""
    N = theta_["N"]
    eta = theta_["eta"]
    S0 = jnp.round(N * eta)
    I0 = 1.0
    R0 = jnp.round(N * (1 - eta)) - 1.0
    H0 = 0.0
    return {"S": S0, "I": I0, "R": R0, "H": H0}
```

```python
def rproc(X_, theta_, key, covars, t, dt):
    """Process simulator for SIR model."""
    S = jnp.asarray(X_["S"])
    I = jnp.asarray(X_["I"])
    R = jnp.asarray(X_["R"])
    H = jnp.asarray(X_["H"])
    Beta = theta_["Beta"]
    mu_IR = theta_["mu_IR"]
    N = theta_["N"]

    p_SI = 1.0 - jnp.exp(-Beta * I / N * dt)
    p_IR = 1.0 - jnp.exp(-mu_IR * dt)
```

```
    key_SI, key_IR = jax.random.split(key)
    dN_SI = jax.random.binomial(key_SI, n=jnp.int32(S), p=p_SI)
    dN_IR = jax.random.binomial(key_IR, n=jnp.int32(I), p=p_IR)

    return {"S": S - dN_SI, "I": I + dN_SI - dN_IR,
            "R": R + dN_IR, "H": H + dN_IR}
```

```
def dmeas(Y_, X_, theta_, covars, t):
    """Measurement density: log P(reports | H, rho, k)."""
    rho = theta_["rho"]
    k = theta_["k"]
    H = X_["H"]
    mu = rho * H
    return nbinom_logpmf(Y_["reports"], k, mu)

def rmeas(X_, theta_, key, covars, t):
    """Measurement simulator."""
    rho = theta_["rho"]
    k = theta_["k"]
    H = X_["H"]
    mu = rho * H
    reports = rnbinom(key, k, mu)
    return jnp.array([reports])
```

Create POMP Object

```
theta = {
    "Beta": 15.0,    # Transmission rate (per week)
    "mu_IR": 0.5,    # Recovery rate (per week)
    "N": 38000.0,    # Population size
    "eta": 0.06,     # Initial susceptible fraction
    "rho": 0.5,      # Reporting probability
    "k": 10.0        # Overdispersion parameter
}

statenames = ["S", "I", "R", "H"]

measSIR = pp.Pomp(
    rinit=rinit,
    rproc=rproc,
    dmeas=dmeas,
    rmeas=rmeas,
    ys=ys,
    theta=theta,
```

```
    statenames=statenames,
    t0=0.0,
    nstep=7,
    accumvars=("H",),
    ydim=1,
    covars=None
)
```

```
W0314 14:43:21.189356 28117753 cpp_gen_intrinsics.cc:74] Empty bitcode string provided for eig
```

Basic Particle Filter

In `pypomp`, the particle filter is implemented via the `pfilter` method. We must choose the number of particles to use by setting the `J` argument.

The `pfilter` method updates the model's `results_history` attribute with the results.

```
# Run a single particle filter
key = jax.random.key(42)
measSIR.pfilter(key=key, J=5000, reps=1)

# Access results from results_history
result = measSIR.results_history.last()
loglik = float(result.logLiks.values[0, 0])
print(f"Log-likelihood: {loglik:.4f}")
```

```
Log-likelihood: -131.4215
```

We can run multiple particle filters to get an estimate of the Monte Carlo variability:

```
# Run 10 replicates of the particle filter
key = jax.random.key(652643293)
measSIR.pfilter(key=key, J=5000, reps=10)

# Get results
result = measSIR.results_history.last()
logliks = result.logLiks.values[0, :]  # All replicates for first theta
print(f"Log-likelihoods: {logliks}")
print(f"Mean: {np.mean(logliks):.4f}, SE: {np.std(logliks):.4f}")
```

```
Log-likelihoods: [-133.1564  -133.78828 -135.3216  -132.25987 -132.47931 -134.3295
 -130.68163 -134.23923 -132.26183 -134.54196]
Mean: -133.3060, SE: 1.3236
```

The logmeanexp Function

To combine multiple log-likelihood estimates, we use the `logmeanexp` function from `pypomp`, which computes:

$$\log\left(\frac{1}{n}\sum_{i=1}^{n} e^{x_i}\right)$$

in a numerically stable way.

```
# pypomp provides logmeanexp and logmeanexp_se
ll_est = pp.logmeanexp(logliks)
ll_se = pp.logmeanexp_se(logliks)
print(f"Log-likelihood estimate: {ll_est:.4f} (SE: {ll_se:.4f})")
```

```
Log-likelihood estimate: -132.4008 (SE: 0.7009)
```

Alternatively, use the `to_dataframe()` method which automatically applies `logmeanexp`:

```
# Get results as DataFrame with logmeanexp already applied
df = result.to_dataframe()
print(df)
```

```
       logLik        se  Beta  mu_IR        N   eta  rho     k
0 -132.400791  0.700865  15.0    0.5  38000.0  0.06  0.5  10.0
```

Maximum Likelihood Estimation

A maximum likelihood estimate (MLE) is:

$$\hat{\theta} = \arg\max_{\theta} \ell(\theta),$$

where $\arg\max_\theta g(\theta)$ means the value of θ at which the maximum of g is attained.

Standard Errors for the MLE

There are three main approaches:

1. **Fisher information**: Computationally quick but often unreliable for POMP models
2. **Profile likelihood estimation**: Generally preferable for POMP models
3. **Bootstrap/simulation study**: Most effort but can be the best approach

Confidence Intervals via Profile Likelihood

Let $\theta = (\phi, \psi)$, where we want a confidence interval for ϕ.

The **profile log-likelihood** of ϕ is:

$$\ell^{\text{profile}}(\phi) = \max_{\psi} \ell(\phi, \psi).$$

An approximate 95% confidence interval for ϕ is:

$$\left\{\phi : \ell(\hat{\theta}) - \ell^{\text{profile}}(\phi) < 1.92\right\}.$$

This is known as a **profile likelihood confidence interval**. The cutoff 1.92 is derived from Wilks' theorem.

Visualizing the Likelihood Surface

- If Θ is two-dimensional, then the surface $\ell(\theta)$ has features like a landscape.
- Local maxima of $\ell(\theta)$ are **peaks**.
- Local minima are **valleys**.
- Peaks may be separated by a valley or may be joined by a **ridge**.

Key features to notice:

- **Wedge-shaped relationships** between parameters are common in epidemiological models
- **Monte Carlo noise** in likelihood evaluation makes it hard to pick out exactly where the likelihood is maximized
- Nevertheless, major features of the likelihood surface are evident despite the noise

Computing Likelihood Slices

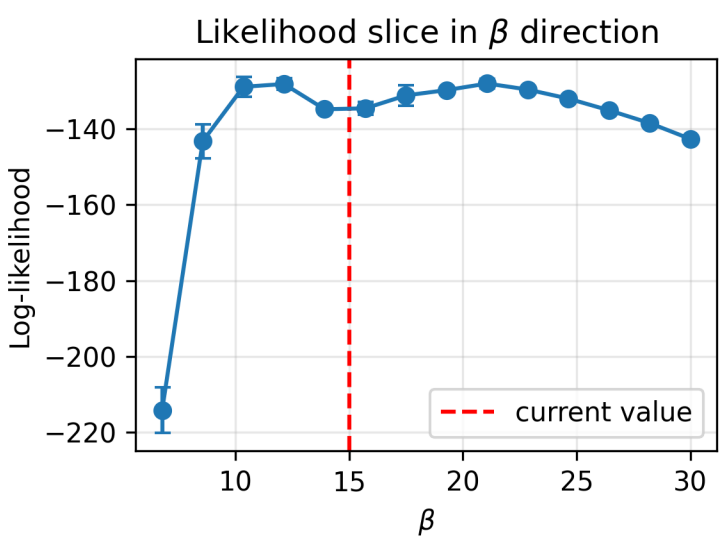
A likelihood slice is a cross-section through the likelihood surface. Let's make slices in the β and μ_{IR} directions.

```
def compute_likelihood_slice(param_name, param_values, base_theta,
                             J=5000, n_reps=3):
    """Compute log-likelihood for a slice of parameter values."""
    results = []
    for val in param_values:
        theta_test = base_theta.copy()
        theta_test[param_name] = val

        pomp_test = pp.Pomp(
            rinit=rinit, rproc=rproc, dmeas=dmeas, rmeas=rmeas,
            ys=ys, theta=theta_test, statenames=statenames,
            t0=0.0, nstep=7, accumvars=("H",), ydim=1, covars=None
        )

        key = jax.random.key(int(val * 1000))
        pomp_test.pfilter(key=key, J=J, reps=n_reps)

        pf_result = pomp_test.results_history.last()
        logliks_arr = pf_result.logLiks.values[0, :]
```

```
        results.append({
            param_name: val,
            'loglik': pp.logmeanexp(logliks_arr),
            'loglik_se': pp.logmeanexp_se(logliks_arr)
        })

    return pd.DataFrame(results)
```

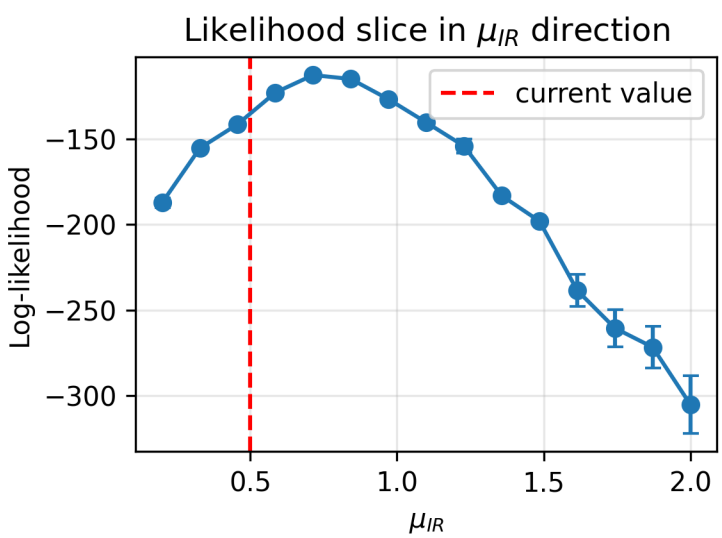
```
# Compute Beta slice
beta_values = np.linspace(5, 30, 15)
beta_slice = compute_likelihood_slice(
    "Beta", beta_values, theta, J=2000, n_reps=3
)
```

```
# Plot Beta slice
fig, ax = plt.subplots(figsize=(4, 3))
ax.errorbar(beta_slice["Beta"], beta_slice["loglik"],
            yerr=beta_slice["loglik_se"], fmt='o-', capsize=3)
ax.axvline(theta["Beta"], color='red', linestyle='--', label='current value')
ax.set(xlabel=r"$\beta$", ylabel="Log-likelihood",
       title=r"Likelihood slice in $\beta$ direction")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



```
# Compute mu_IR slice
mu_IR_values = np.linspace(0.2, 2.0, 15)
mu_IR_slice = compute_likelihood_slice(
    "mu_IR", mu_IR_values, theta, J=2000, n_reps=3
)
```

```
# Plot mu_IR slice
fig, ax = plt.subplots(figsize=(4, 3))
ax.errorbar(mu_IR_slice["mu_IR"], mu_IR_slice["loglik"],
            yerr=mu_IR_slice["loglik_se"], fmt='o-', capsize=3)
ax.axvline(theta["mu_IR"], color='red', linestyle='--', label='current value')
ax.set(xlabel=r"$\mu_{IR}$", ylabel="Log-likelihood",
       title=r"Likelihood slice in $\mu_{IR}$ direction")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Two-Dimensional Likelihood Surface

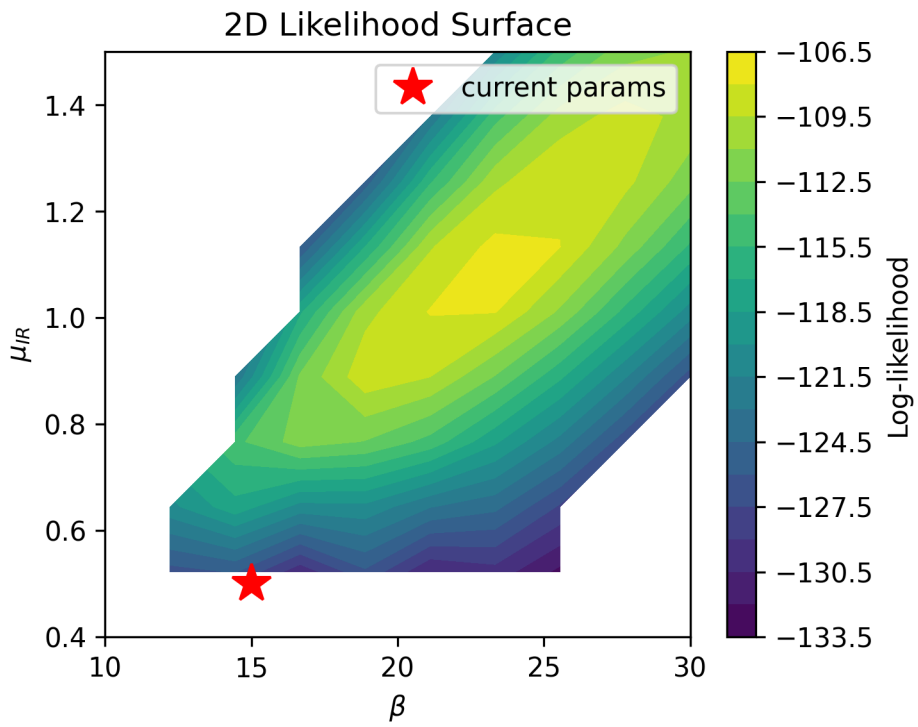
```
def compute_2d_likelihood_surface(beta_vals, mu_IR_vals, base_theta,
                                  J=2000, n_reps=2):
    """Compute 2D log-likelihood surface."""
    results = []
    for beta in beta_vals:
        for mu_IR in mu_IR_vals:
            theta_test = base_theta.copy()
            theta_test["Beta"] = beta
            theta_test["mu_IR"] = mu_IR
```

```
            pomp_test = pp.Pomp(
                rinit=rinit, rproc=rproc, dmeas=dmeas, rmeas=rmeas,
                ys=ys, theta=theta_test, statenames=statenames,
                t0=0.0, nstep=7, accumvars=("H",), ydim=1, covars=None
            )

            key = jax.random.key(int(beta * 100) + int(mu_IR * 1000))
            pomp_test.pfilter(key=key, J=J, reps=n_reps)

            pf_result = pomp_test.results_history.last()
            logliks_arr = pf_result.logLiks.values[0, :]

            results.append({
                'Beta': beta,
                'mu_IR': mu_IR,
                'loglik': pp.logmeanexp(logliks_arr)
            })

    return pd.DataFrame(results)
```

```
# Compute 2D surface (reduced grid for speed)
beta_grid = np.linspace(10, 30, 10)
mu_IR_grid = np.linspace(0.4, 1.5, 10)

surface_df = compute_2d_likelihood_surface(
    beta_grid, mu_IR_grid, theta, J=1000, n_reps=2
)
```

```
# Plot as contour
pivot = surface_df.pivot(index='mu_IR', columns='Beta', values='loglik')

fig, ax = plt.subplots(figsize=(5, 4))
X, Y = np.meshgrid(pivot.columns, pivot.index)
Z = pivot.values

# Mask values too far from maximum
max_ll = np.nanmax(Z)
Z_masked = np.where(Z > max_ll - 25, Z, np.nan)

contour = ax.contourf(X, Y, Z_masked, levels=20, cmap='viridis')
plt.colorbar(contour, ax=ax, label='Log-likelihood')
ax.set(xlabel=r'$\beta$', ylabel=r'$\mu_{IR}$',
       title='2D Likelihood Surface')
ax.plot(theta["Beta"], theta["mu_IR"], 'r*', markersize=15,
        label='current params')
```

```
ax.legend()
plt.tight_layout()
plt.show()
```



Maximizing the Particle Filter Likelihood

- Likelihood maximization is key to profile intervals, likelihood ratio tests, and AIC, as well as computation of the MLE.
- An initial approach might be to use the particle filter log-likelihood estimate with a standard numerical optimizer (e.g., Nelder-Mead).
- In practice, this approach is unsatisfactory on all but the smallest POMP models.
- Standard numerical optimizers are not designed to maximize **noisy** and **computationally expensive** Monte Carlo functions.

Trade-offs:

- If we use a deterministic optimizer and fix the RNG seed, the objective function becomes **jagged** (many small local knolls and pits).
- If we use a stochastic optimization algorithm, we can only obtain **estimates** of the MLE.

We'll present **iterated filtering** in the next lesson as a better approach.

Likelihood Ratio Tests for Nested Hypotheses

Suppose we have two nested hypotheses:

- $H^{\langle 0 \rangle}$: $\theta \in \Theta^{\langle 0 \rangle}$ (dimension $D^{\langle 0 \rangle}$)
- $H^{\langle 1 \rangle}$: $\theta \in \Theta^{\langle 1 \rangle}$ (dimension $D^{\langle 1 \rangle}$)

where $\Theta^{\langle 0 \rangle} \subset \Theta^{\langle 1 \rangle}$.

Wilks' approximation: Under the null hypothesis $H^{\langle 0 \rangle}$:

$$\ell^{\langle 1 \rangle} - \ell^{\langle 0 \rangle} \approx \frac{1}{2}\chi^2_{D^{\langle 1 \rangle} - D^{\langle 0 \rangle}}$$

This can be used to construct a **likelihood ratio test**.

Akaike's Information Criterion (AIC)

For non-nested hypotheses, we can compare likelihoods using AIC:

$$\text{AIC} = -2\ell(\hat{\theta}) + 2D$$

"Minus twice the maximized log-likelihood plus twice the number of parameters."

- Select the model with the **lowest AIC** score.
- AIC was derived as an approach to minimizing prediction error.
- Increasing parameters leads to overfitting which can decrease predictive skill.

Practical guidance:

- AIC is useful for selecting a model with reasonable predictive skill from a range of possibilities.
- View it as a procedure to select a reasonable predictive model, not as a formal hypothesis test.
- BIC provides a more severe penalty for complexity.

Exercise 3.1: Slices and Profiles

What is the difference between a likelihood **slice** and a **profile**? What is the consequence of this difference for the statistical interpretation of these plots? How should you decide whether to compute a profile or a slice?

Exercise 3.2: Cost of a Particle Filter Calculation

- How much computer processing time does a particle filter take?
- How does this scale with the number of particles?

Form a conjecture based upon your understanding of the algorithm. Test your conjecture by running a sequence of particle filter operations, with increasing numbers of particles (J), measuring the time taken for each one. Plot and interpret your results.

Exercise 3.3: Log-likelihood Estimation

Here are some desiderata for a Monte Carlo log-likelihood approximation:

- It should have low Monte Carlo bias and variance.
- It should be presented together with estimates of the bias and variance so that we know the extent of Monte Carlo uncertainty in our results.
- It should be computed in a length of time appropriate for the circumstances.

Set up a likelihood evaluation for the measles model, choosing the numbers of particles and replications so that your evaluation takes approximately one minute on your machine. Provide a Monte Carlo standard error for your estimate. Comment on the bias of your estimate.

Exercise 3.4: One-dimensional Likelihood Slice

Compute several likelihood slices in the η direction.

Exercise 3.5: Two-dimensional Likelihood Slice

Compute a slice of the likelihood in the β-η plane.

Summary

1. The **likelihood function** is central to frequentist, Bayesian, and information-theoretic inference.
2. For POMP models, the likelihood involves a high-dimensional integral that cannot be computed analytically.
3. The **particle filter** provides an efficient Monte Carlo algorithm for computing the likelihood:
   - Alternates between prediction (simulation) and filtering (resampling)
   - Provides an unbiased estimate of the likelihood
4. **Likelihood-based inference** provides tools for:
   - Maximum likelihood estimation
   - Profile likelihood confidence intervals
   - Likelihood ratio tests
   - Model comparison via AIC
5. The **geometry of the likelihood surface** reveals important features:
   - Wedge-shaped relationships between parameters
   - Monte Carlo noise affects optimization

References

Fisher, R. A. (1922). On the mathematical foundations of theoretical statistics. *Philos Trans R Soc London A*, 222:309–368.